实验系统设计规范书：多轮对话风格漂移控制代理

版本: 1.2 (Final Revised) **依据文档**:

1. `Project_description.docx`（理论基础 & 角色定义）
2. `Engineering and Development Procedure.docx`（工程流程 & 数据契约）
3. `Control Agent System Prompt.docx`（执行逻辑 & 错误处理细节）

适用对象: 工程开发团队、研究人员

1. 系统概述 (System Overview)

本系统构建一个**自动化**、**确定性**、**可复现**的多智能体控制架构。 **核心目标**: 模拟用户（LLM2脚本）与被测模型（LLM3）在不同系统提示词（Default/Evil/Distant）下的多轮交互，并由裁判模型（LLM4）实时打分，最终生成用于分析“风格漂移”及“回归安全均值”现象的**轮次级 JSONL 数据集**。

核心职责: 编排（Orchestration）与日志记录（Logging）。系统**不**负责生成对话内容（由模型负责）或数据分析。

2. 系统架构与角色定义 (Architecture & Roles)

2.1 离线准备角色 (Offline / Frozen)

*此部分产出物在实验运行时必须保持**不可变（Immutable）**。*

- **LLM1 - 提示设计师 (Prompt Designer)**:
  - **功能**: 生成极端风格（Evil/Distant）的系统提示词草稿。
  - **产出**: 经人工审核后冻结在 `prompts.json` 中。
- **LLM2 - 剧本生成器 (Dialogue Generator)**:
  - **功能**: 生成 2,000 个“仅用户（User-only）”对话脚本，平衡分布于 Creative, Finance, Mental Health, Medicine 四个领域。
  - **产出**: 存储于 `dialogues.jsonl` 中。

2.2 在线运行角色 (Runtime Execution)

- **LLM3 - 生成器 (Generator / Actor)**:
  - **功能**: 被测模型。根据系统提示词 + 历史上下文生成回复。
  - **要求**: 支持主流 LLM API (OpenAI 等)，参数 (Temp, Top_p) 可配置。
- **LLM4 - 裁判 (Judge / Evaluator)**:
  - **功能**: 对 LLM3 的**单轮回复**进行评分。
  - **维度**: Harmfulness, Emotionality, Anthropomorphism (1-5分)。
  - **约束**: 输出严格遵循 JSON Schema。

- **Control Agent - 控制代理 (Orchestrator)**:
  - **功能**: 状态机。负责读取输入、顺序调用 LLM3/LLM4、处理错误、写入日志。

3. 输入数据契约 (Input Contracts)

3.1 `dialogues.jsonl` (权威脚本)

- **结构**: JSON Lines。
- **必需字段**:
  - `dialogue_id` (唯一标识)
  - `domain` (枚举: `creative` , `finance` , `mental_health` , `medicine`)
  - `turns` (数组: 规范格式为 `[{"role": "user", "text": "..."}]`)
- **兼容模式 (Optional)**: 若 `turns` 格式非规范（如仅字符串列表），且配置开启兼容模式，加载器需自动转换并记录 `input_schema_variant` 。
- **处理规则**: 加载时忽略任何非 `user` 角色的内容。

3.2 `prompts.json` (提示词库)

- **必需字段**:
  - `conditions` : 字典，包含 `default` , `evil` , `distant` 三个 Key 对应的 System Prompt 字符串。
  - `judge_system` : 裁判的 System Prompt。
  - `judge_rubric` : 评分标准文本。
  - `judge_schema` : 严格的 JSON Schema 用于约束裁判输出。
- **约束**: 运行时**严禁**修改或“优化”这些提示词。

3.3 `config.yaml` (运行时配置)

- **模型配置**: LLM3 和 LLM4 的 Model ID、Temperature、Top_p、Seed。
- **实验配置**: `max_turns` , `output_dir` , `resume_strategy` (skip/reconstruct), `flush_policy` 。
- **运维配置**: `retries` , `timeout` , `truncation_policy` , `abort_on_error` (默认 False)。
- **安全**: API Key 必须从环境变量读取。

4. 核心执行逻辑 (Core Execution Logic)

4.1 循环层级 (The Loop)

1. **初始化**: 生成唯一的 `run_id` ，计算所有输入文件 (`prompts` , `dialogues` , `config`) 的 Hash 值用于溯源。
2. **外层循环 (Dialogues)**: 按**确定性顺序**（文件顺序或 ID 排序）遍历对话。
3. **中层循环 (Conditions)**: **强制固定顺序**: `default` → `evil` → `distant` 。

- *关键点*: 切换条件时，**必须重置**对话历史（仅保留该条件的 System Prompt）。

4. **内层循环 (Turns)**: 遍历用户轮次 (1...T)。

4.2 单轮处理流程 (Per-Turn Workflow)

1. **构造输入**: `System Prompt + History + Current User Turn` 。
   - **应用截断**: 根据配置（策略A: 滑动窗口 或 策略B: Token预算）处理过长的历史，并标记 `context_truncated=true` 。
2. **调用 LLM3 (Generate)**:
   - **成功**: 获取 `model_reply` 。
   - **拒答 (Refusal)**: 若 API 返回 200 但内容是拒答，视为**正常生成**，记录 `refusal_detected=true` ，继续流程。
   - **失败 (API Error)**: 记录 `error_stage="generate"` ，**不**追加历史，跳过后续步骤，进入下一轮（除非配置了 `abort_on_error=true` ）。
3. **调用 LLM4 (Judge)**:
   - **输入**: 仅 `Judge System Prompt + Rubric + Model Reply`（减少干扰）。
   - **输出**: 解析 JSON 分数。
   - **失败**: 记录 `error_stage="judge"` 或 `"judge_parse"` ，分数为 null。
4. **写入日志**: 立即 Flush 一行 JSON 到磁盘。
5. **更新历史**: 将 `User Turn` 和 `Model Reply` 追加到内存历史中。
   - *注意*: 仅当 LLM3 API 调用成功时追加。

4.3 断点续传 (Resume Behavior)

- **必须支持**。配置项 `resume_strategy` ：
  - `"reconstruct"` （推荐): 读取现有日志，重建内存中的对话历史，从断点处继续。
  - `"skip"` ：简单跳过已存在的 `(dialogue_id, condition)` 组合。

5. 数据输出规范 (Output Specification)

5.1 文件契约

- **路径**: `output_dir/results_{run_id}.jsonl`
- **格式**: JSON Lines，UTF-8，一行一轮。

5.2 字段定义 (Schema)

每行必须包含以下字段：

类别	字段名	说明
元数据	`run_id`	全局唯一运行 ID

	`dialogue_id`	对应输入文件的 ID
	`domain`	领域
	`condition`	`default` / `evil` / `distant`
	`turn_index`	当前轮次 (1-based)
	`timestamp_utc`	ISO 8601 时间戳
内容	`user_text`	用户输入
	`model_reply`	LLM3 回复 (生成失败则为空字符串)
评分	`harm` , `emotion` , `anthro`	1-5 整数 (失败则为 null)
诊断	`error_stage`	`null` , `"generate"` , `"judge"` , `"judge_parse"`
	`error_message`	具体的错误信息
	`refusal_detected`	`true` / `false` (启发式检测)
	`context_truncated`	`true` / `false` (是否触发截断)
	`gen_latency_ms`	LLM3 生成耗时 (毫秒)
	`judge_latency_ms`	LLM4 评分耗时 (毫秒)
溯源	`llm3_model` , `llm3_params`	模型名及参数快照
	`resume_strategy`	当前运行使用的续传策略
	`prompts_hash`	`prompts.json` 文件哈希
	`config_hash`	`config.yaml` 文件哈希
	`dialogues_hash`	`dialogues.jsonl` 文件哈希

6. 错误处理与质量保障 (QA & Error Handling)

6.1 错误处理策略

- **API 故障 (5xx/429/Timeout)**:
  - 执行指数退避重试 (Exponential Backoff)。

    - 达到最大重试次数后：记录错误， `model_reply` 置空，**切勿**将空回复追加到对话历史（防止污染上下文），继续下一轮。
- **解析故障 (JSON Parse Error)**:
    - 记录原始返回内容（ `judge_raw` ）到日志以便审计。
    - 分数字段置为 `null` 。

6.2 质量检查 (Checklist)

1. **运行前 (Pre-run)**:
    - 校验 `dialogues.jsonl` Schema 及唯一性。
    - 校验 `prompts.json` 完整性。
2. **冒烟测试 (Smoke Test)**:
    - 支持 CLI 参数 `--dry_run` ，仅运行前 5 个对话，快速验证 API 连通性和日志格式。
3. **运行后 (Post-run)**:
    - 必须生成 `run_summary_{run_id}.json` 。
    - 统计内容：预期行数 vs 实际行数、错误率、截断次数、拒答率、所有输入文件的 Hash 快照。

7. 交付清单 (Deliverables)

1. **可执行程序 (CLI)**: `python control_agent.py --config ...`
2. **验证报告**: 包含一次 `--dry_run` 的日志和 Summary。